

Azure Cloud Mentorship Projects

Phase 1: AZ-104 (Microsoft Azure Administrator) – 3 Months

Month 1: Company – FinTechEdge

Project Name on GitHub:

[FinTechEdge-Terraform-VMSS-Traffic-Manager-AzureFirewall](#)

FinTechEdge needs a resilient, **scalable infrastructure on Azure to host their customer-facing web application, expected to see high traffic**. They require a secure setup with Infrastructure as Code (IaC) to manage deployments and ensure compliance with organizational policies. The setup will need cost monitoring and alerting mechanisms, automated backups, disaster recovery, and ongoing resource monitoring.

Project Objective

Deploy a fully functional Azure infrastructure meeting the following objectives:

- **Identity & Access Management:** Use Azure AD, Role-Based Access Control (RBAC), and implement Multi-Factor Authentication (MFA).
- **Compute & Networking:** Set up VM Scale Sets (VMSS) and load balancing across two regions.
- **Storage & Data Protection:** Configure Azure Blob Storage and backups.
- **Monitoring:** Enable Azure Monitor, Log Analytics, and set up alerts.
- **Automation:** Use Azure Automation and CLI for resource management.
- **Governance & Compliance:** Apply policies with Azure Policy, cost management, and set up a budget.

Key Deliverables

1. **Terraform scripts** for the entire infrastructure.
2. **GitHub Repository** with all code, README, and instructions.
3. **Documentation** on setup and management of the environment, troubleshooting, and governance.
4. **Screenshots** for proof of setup, backup, automation, failover testing, and policy enforcement.

Step-by-Step Project Setup

Step 1: Set Up Identity & Access Management

1. **Create Azure AD Users & Groups:**
 - Define AAD users and groups for different roles (e.g., Admins, Developers, and Viewers).
 - Configure RBAC roles to restrict access based on user roles.
2. **Enable Multi-Factor Authentication (MFA):**
 - Use Conditional Access policies to require MFA for critical access (e.g., administrative operations).

Step 2: Configure Virtual Networks, VM Scale Sets, and Load Balancing

1. **Define Virtual Networks:**
 - Create two virtual networks in different regions (e.g., East US and West US) using Terraform.
 - Implement subnets and Network Security Groups (NSGs) to manage traffic between the application and data subnets.
2. **Deploy VM Scale Sets (VMSS):**
 - Set up VMSS in each region with autoscaling based on CPU usage.
 - Attach VMSS instances to a Load Balancer in each region.
3. **Azure Traffic Manager:**
 - Configure failover routing between East and West regions using Traffic Manager for global traffic management.

Step 3: Configure Storage and Data Protection

1. **Set Up Azure Blob Storage:**
 - Create a storage account with Blob Storage for application data.
 - Enable geo-redundancy for high availability.
2. **Automate Backups with Azure Backup:**
 - Use Azure Backup to create backups for VMs and Blob storage.
 - Schedule backups and configure retention policies.
3. **Configure Disaster Recovery:**
 - Set up Azure Site Recovery to replicate VMs to the secondary region for failover.

Step 4: Security & Compliance

1. **Deploy Azure Firewall and Network Security Groups (NSGs):**
 - Set up an Azure Firewall to filter inbound and outbound traffic.
 - Use NSGs to manage traffic rules within subnets, securing communication between app components.
2. **Apply Azure Policy for Governance:**
 - Enforce policies to ensure compliance, e.g., restrict regions, enforce tag usage, and limit VM SKU availability.
 - Create a Blueprint for consistent deployments across environments.

Step 5: Monitoring and Alerts

1. **Configure Azure Monitor and Log Analytics:**
 - Set up Azure Monitor for infrastructure insights.
 - Create Log Analytics workspaces and configure diagnostic settings for VMs, storage, and network resources.
2. **Set Up Alerts for Resource Health:**
 - Define alerts based on CPU usage, disk space, and network latency.
 - Use Action Groups to send alerts to admins.
3. **Use Application Insights:**
 - Enable Application Insights to monitor application performance and capture errors for the web app.

Step 6: Implement Automation

1. **Create Azure Automation Account:**
 - Set up an Automation Account to run scripts and manage resources.
 - Use runbooks to automate daily health checks and VM start/stop schedules for cost savings.
2. **Implement Azure CLI Commands in Runbooks:**
 - Automate common tasks like VM scaling, backup verification, and status checks using CLI within runbooks.

Step 7: Cost Management

1. **Set Up Cost Management:**
 - Use Azure Cost Management to track usage and spending.
 - Configure budgets and set alerts for spending thresholds.
2. **Enable Resource Tagging:**
 - Apply tags for environment (e.g., Prod, Dev), department, and cost center to track costs effectively.

Step 8: Version Control with GitHub

1. **Create GitHub Repository:**
 - Use a repository structure with directories for `terraform`, `scripts`, and `docs`.
 - Add a detailed `README.md` with instructions on setup, deployment, and troubleshooting.
2. **Push Terraform Code and Documentation:**
 - Commit all Terraform scripts, configurations, and screenshots with clear commit messages.
 - Document the project with deployment steps, screenshots, and notes on automation, governance, and cost management.

Acceptance Criteria

1. **Version Control:**

- All Terraform scripts, configuration files, and screenshots are committed to GitHub with clear commits and branches.
- 2. **Documentation:**
 - README.md contains setup instructions, deployment steps, troubleshooting tips, and project insights.
- 3. **Testing and Validation:**
 - Screenshots showing:
 - Identity management with Azure AD users and RBAC roles.
 - VM Scale Sets, Traffic Manager failover, and Load Balancer in action.
 - Successful backups and Azure Site Recovery configured.
 - Firewall and NSG rules blocking and allowing expected traffic.
 - Cost management alerts, budgets, and tagged resources.
- 4. **Automation Verification:**
 - Runbooks in the Azure Automation account executing scheduled tasks.
 - Alerts and notifications triggering successfully for monitoring.

Expected Outcome

FinTechEdge will have a resilient, secure, and compliant Azure environment with:

- Automated infrastructure deployment and management via Terraform.
- Identity, access, and network security configured.
- Storage and disaster recovery for data protection.
- Monitoring, alerting, and cost management.

Month 2: Company – CloudSecureIT

Project Name on GitHub:

CloudSecureIT-Docker-ACR-WebApp-Certificates

Project Overview

CloudSecureIT is developing a **secure document storage web application** that allows authenticated users to upload and retrieve documents through a web-based interface. The application will be **containerized using Docker** and deployed on **Azure Web Apps**, with **Azure Blob Storage** managing document storage.

Users must log in to the system before uploading files, and after a successful upload, they will receive a **secure hyperlink** to access their document later.

Features & Functionality

1 Web-Based Document Submission Form

Users must fill out a form with the following fields:

- First Name
- Last Name
- Email Address
- Document Type (Dropdown: Resume, Project, Profile, Script)
- File Upload (PDF, DOCX, TXT, etc.)
- **Submit Button**

Upon submission, the user will see a message:

“Thank you for your submission. We will contact you shortly.”

The system generates a **hyperlink** that allows the user to access their document securely.

2 Secure File Storage with Azure Blob Storage

- Files are uploaded and stored securely in **Azure Blob Storage**.
- Documents are categorized based on the selected document type.
- Users receive a **securely generated hyperlink** to retrieve their document.

3 User Authentication (User Auth Method)

- Only **authenticated users** can upload documents.
- Authentication options can include:
 - **Azure Active Directory (AAD) B2C**
 - **OAuth2 / JWT Authentication**
 - **Custom User Management System (MongoDB/PostgreSQL)**

4 Docker & Deployment to Azure

- The entire application will be **containerized using Docker**.
- Uses **Azure Container Registry (ACR)** to store Docker images.
- Deployed to **Azure Web Apps** for hosting.
- **SSL/TLS encryption** ensures secure web access.

Deliverables

✓ **Node.js Web Application** with:

- Express.js backend to handle authentication and file uploads.
- Frontend with HTML/CSS for the document submission form.

✓ Dockerized Application

- **Dockerfile** for building the Node.js app.
- Pushes Docker image to **Azure Container Registry (ACR)**.

✓ Terraform Scripts for Infrastructure

- Deploys **Azure Blob Storage** for file storage.
- Creates **Azure Web App** linked to **ACR**.
- Sets up **SSL/TLS certificate** for secure access.

✓ GitHub Repository Structure

- `/src` → Node.js backend
- `/public` → Frontend form (HTML/CSS)
- `/terraform` → Infrastructure as Code
- `README.md` → Deployment & usage instructions

✓ Testing & Validation

- **Successful HTTPS deployment** on Azure Web Apps.
- **Test file uploads** and retrieval via secure hyperlinks.
- **User authentication before access is granted.**

Acceptance Criteria

1. Version Control:

- GitHub repository exists and follows a proper branching strategy.
- All Dockerfiles, Terraform scripts, and configurations are committed.

2. Documentation (README.md):

- Clear instructions on **building and pushing** the Docker image to ACR.
- Steps for **deploying the app** to Azure Web Apps using Terraform.
- Setup details for **SSL/TLS certificates** and authentication.
- Instructions on configuring **Azure Blob Storage** for document storage.
- Screenshots of:
 - **Successful Docker image push to ACR.**
 - **Azure Web App deployment from ACR.**
 - **SSL/TLS certificate configuration.**
 - **File uploads working securely in Blob Storage.**

3. Functional Testing:

- Users can **log in** using the authentication system.
- Users can **upload documents** successfully.

- Documents are **stored in Azure Blob Storage** securely.
 - The app **returns a secure hyperlink** to the user for accessing their document.
4. **Infrastructure Setup:**
- Docker-based deployment with **Azure Web Apps**.
 - Secure **Azure Blob Storage** integration.
 - **SSL/TLS certificates** correctly applied.

Expected Outcome:

CloudSecureIT will have a **Dockerized web application** running securely in Azure Web Apps, with **SSL protection** and **secure file storage** in Azure Blob Storage. All infrastructure and deployment processes will be documented and managed via **Terraform** and GitHub.

Month 3: Company – DataBridge Analytics

Project Name on GitHub:

[DataBridge-VMSS-Monitoring-Blob-Storage-DisasterRecovery](#)

Problem Faced:

DataBridge Analytics, a fintech company providing online financial services, is launching a new containerized web application for their customers. This app is expected to handle high traffic while maintaining strict data security and compliance with financial regulations. FinServe Inc. requires an automated deployment pipeline, secure access controls, real-time monitoring, and backup processes to ensure service reliability and resilience.

Problem Faced

FinServe Inc. is preparing to deploy their containerized web application in Azure to support a growing user base. They face several key challenges:

1. **Automated and Secure Deployment:** They need a continuous deployment (CI/CD) pipeline to ensure smooth updates to the app with secure access to sensitive configuration information.
2. **Environment Segmentation:** FinServe requires separate staging and production environments to validate changes before releasing to end users.
3. **Data Security:** They must secure sensitive financial data, control access to resources, and ensure regulatory compliance.
4. **Monitoring and Incident Response:** Real-time monitoring, logging, and alerting are essential for detecting issues and responding to incidents quickly.
5. **Disaster Recovery:** Automated data backup and recovery plans are required to protect against data loss or service interruptions.

Project Objective

Build a secure, automated, and scalable infrastructure on Azure to deploy and manage the FinServe app. The infrastructure should include:

1. **CI/CD pipeline** for consistent deployments.
2. **Staging and production environments** to validate app changes.
3. **Robust security controls** to manage access and protect sensitive information.
4. **Comprehensive monitoring and alerting** for operational visibility.
5. **Automated backup** to safeguard critical data and support disaster recovery.

Project Goals

1. **Automate Application Deployment:**
 - Use GitHub Actions to automate deployment to Azure App Service.
 - Configure deployment slots for staging and production environments.
2. **Ensure Data Security and Access Control:**
 - Securely manage secrets using Azure Key Vault.
 - Implement Conditional Access and Just-In-Time (JIT) access for enhanced security.
 - Apply Network Security Groups (NSGs) to restrict traffic.
3. **Enable Monitoring and Alerting:**
 - Configure Azure Monitor and Log Analytics to capture application logs and metrics.
 - Create custom alerts and Action Groups to notify the operations team of critical issues.
4. **Automate Data Backup and Disaster Recovery:**
 - Set up automated backups for critical storage and configuration data.
 - Document and test recovery procedures for business continuity.
5. **Maintain Compliance and Governance:**
 - Use Azure Secure Score to continuously monitor and improve the security posture.
 - Enforce tagging policies and access controls to align with governance requirements.

Project Deliverables

1. **GitHub Repository:**
 - Contains GitHub Actions workflows for CI/CD, Terraform scripts for infrastructure deployment, and README documentation with setup instructions.
2. **Infrastructure Components:**
 - Azure App Service with staging and production slots.
 - Azure Container Registry (ACR) for secure image storage.
 - Azure Key Vault for sensitive data management.
 - NSGs and Conditional Access configured for secure access control.
3. **Monitoring and Alerts:**

- Azure Monitor with Log Analytics for tracking performance metrics and application logs.
- Custom alerts for critical metrics with Action Groups for notifications.
- 4. **Automated Backup and Disaster Recovery:**
 - Scheduled backups for storage accounts and configurations.
 - Documented disaster recovery plan with backup verification steps.

Acceptance Criteria

1. **Automated Deployment Pipeline:**
 - GitHub Actions pipeline successfully deploys the application to staging and production slots on Azure App Service.
 - Deployment pipeline securely accesses secrets stored in Azure Key Vault.
2. **Environment Security:**
 - NSGs are applied to restrict access to the application and supporting services.
 - Conditional Access policies enforce multi-factor authentication (MFA) for privileged users.
 - Just-In-Time (JIT) access is enabled on VMs for administrative access.
3. **Operational Monitoring and Alerts:**
 - Azure Monitor and Log Analytics capture key metrics, with custom alerts configured for high CPU usage, error rates, and unauthorized access attempts.
 - Action Groups notify the operations team of critical alerts, and Logic Apps automate basic incident responses.
4. **Data Protection and Disaster Recovery:**
 - Automated backups are configured for critical resources, and data recovery has been tested and verified.
 - Backup verification scripts run periodically to confirm data integrity.
5. **Compliance and Governance:**
 - Azure Secure Score is monitored with improvements documented.
 - Resource tagging and access policies are enforced to meet governance requirements.

Expected Outcome

FinServe Inc. will have a robust, secure, and automated Azure infrastructure to support their containerized application with the following benefits:

1. **Reliability:** The CI/CD pipeline ensures consistent, secure deployments across staging and production, reducing the risk of downtime during updates.
2. **Security:** Key Vault, NSGs, Conditional Access, and JIT access protect sensitive data and secure the infrastructure.
3. **Operational Visibility:** Real-time monitoring, logging, and alerts provide insights into application performance and security events, enabling quick response to issues.

4. **Data Integrity:** Automated backups and disaster recovery plans ensure data protection and business continuity.
5. **Compliance:** Continuous Secure Score monitoring and documented governance practices help FinServe meet regulatory standards.

Midterm Project for AZ-104: FinTechEdge Full Cloud Solution

Project Name on GitHub:

FinTechEdge-Full-Cloud-Solution

Problem Faced:

DataFortify Inc. needs to deploy a scalable, multi-tier data analytics application with a high degree of security, compliance, and automation. The company expects this application to handle sensitive customer data, requiring stringent data protection, robust access controls, and automated monitoring for continuous security and operational efficiency.

DataFortify Inc. faces the following challenges:

1. **Complex Networking Requirements:** The application requires a multi-region setup with virtual network peering, load balancing, and secure access controls for internal and external resources.
2. **Data Protection and Compliance:** Sensitive data needs to be secured with encryption, regular backups, and managed identities for service communication without exposing secrets.
3. **Advanced Monitoring and Alerting:** Real-time monitoring and alerting for performance, security events, and compliance.
4. **Automation and Governance:** Automated provisioning, backup, and remediation of policy violations to maintain compliance and reduce manual intervention.

Project Objective

Deploy a secure, multi-tier infrastructure for DataFortify's application, focusing on:

1. **Multi-Region Networking and Load Balancing:** Use Virtual Network Peering, Private Link, and Application Gateway.
2. **Data Protection and Secure Access:** Implement encryption, Azure Key Vault, Managed Identity, and Private Endpoints.
3. **Continuous Monitoring and Compliance:** Configure Azure Monitor, Log Analytics, and automated alerting with custom action groups.

4. **Automated Backups and Remediation:** Set up Azure Backup and use Azure Automation and Policy to enforce compliance and reduce manual tasks.

Project Goals

1. **Advanced Networking and High Availability:**
 - Set up Virtual Network Peering for multi-region connectivity.
 - Use Application Gateway and Load Balancer to distribute traffic.
 - Secure data with Private Link and Service Endpoints to prevent public exposure.
2. **Secure Identity and Access Management:**
 - Use Managed Identity for secure, passwordless communication between services.
 - Configure Network Security Groups (NSGs) and Conditional Access for network traffic control.
 - Use Just-In-Time (JIT) access to secure administrative VM access.
3. **Comprehensive Monitoring and Alerts:**
 - Implement Azure Monitor and Log Analytics for real-time data collection.
 - Set up custom alerts for specific metrics (e.g., VM CPU usage, network latency).
 - Define Action Groups and Logic Apps for automated incident response.
4. **Automated Backups and Compliance:**
 - Schedule automated backups for storage and database resources.
 - Configure Azure Policy and Automation to enforce compliance (e.g., tagging, secure protocols).
 - Implement Runbooks for tasks like backup verification and resource shutdown during off-hours.

Project Deliverables

1. **GitHub Repository:**
 - Contains all Terraform scripts, Runbooks, and documentation.
 - README with setup instructions, troubleshooting steps, and security configurations.
2. **Infrastructure Components:**
 - Azure Virtual Networks (peered), Load Balancer, and Application Gateway.
 - Azure Key Vault, Private Link, and Managed Identity for secure access.
 - NSGs, Conditional Access policies, and Just-In-Time (JIT) access configuration.
3. **Monitoring and Automation:**
 - Log Analytics Workspace for aggregated log data.
 - Custom alerts, Action Groups, and Logic Apps for incident response.
 - Automated backup schedules and Runbooks for routine tasks.
4. **Security and Compliance Reports:**
 - Secure Score dashboard for continuous monitoring.
 - Documentation of Azure Policy compliance and remediation.

Step-by-Step Setup

Step 1: Multi-Region Networking Setup

1. **Virtual Networks (VNet) and Peering:**
 - Create VNets in two regions (e.g., East US and West US) with peering between them for secure cross-region communication.
2. **Application Gateway and Load Balancer:**
 - Deploy an Application Gateway with Web Application Firewall (WAF) to distribute and protect traffic to the application.
 - Use an internal Load Balancer for backend VMs, distributing requests to instances across availability zones.
3. **Private Link and Service Endpoints:**
 - Configure Private Link for resources like Azure SQL Database and Storage Accounts to keep traffic private within the Azure network.
 - Apply Service Endpoints to VNets for additional resources needing secure access.

Step 2: Secure Identity and Access Control

1. **Azure Key Vault and Managed Identity:**
 - Store sensitive data in Azure Key Vault and allow access only through Managed Identities for VMs, App Services, and other Azure resources.
2. **Network Security Groups (NSGs):**
 - Apply NSGs to subnets to restrict inbound and outbound traffic based on IP ranges and protocols.
3. **Conditional Access and JIT Access:**
 - Use Conditional Access for MFA enforcement on admin access to critical resources.
 - Enable Just-In-Time (JIT) access on VMs to secure remote access.

Step 3: Configure Monitoring with Azure Monitor, Log Analytics, and Alerts

1. **Log Analytics Workspace:**
 - Create a Log Analytics Workspace to centralize logs from VMs, Application Gateway, Key Vault, and other resources.
2. **Azure Monitor Metrics and Custom Alerts:**
 - Use Azure Monitor to track application performance metrics.
 - Set up custom alerts for critical performance metrics (e.g., CPU, memory, network latency) and security events (e.g., failed login attempts).
3. **Action Groups and Logic Apps:**
 - Configure Action Groups to send notifications via email, SMS, or webhook.
 - Use Logic Apps for automated responses (e.g., scaling resources, blocking IPs after security events).

Step 4: Data Protection with Automated Backups

1. **Azure Backup for VMs and Databases:**
 - Configure Azure Backup to automate backup schedules for VMs and databases in primary and secondary regions.
 - Set up retention policies and geo-redundancy to support disaster recovery.
2. **Runbook for Backup Verification:**
 - Create a Runbook in Azure Automation to periodically restore and verify backups for data integrity.
3. **Storage Account for Logs and Snapshots:**
 - Configure a Storage Account to hold backup snapshots, logs, and other artifacts for auditing and compliance.

Step 5: Compliance and Governance

1. **Azure Policy and Blueprints:**
 - Define Azure Policies to enforce compliance (e.g., enforced tagging, restricting certain VM SKUs, HTTPS-only endpoints).
 - Use Blueprints to automate the deployment of compliant resources across environments.
2. **Secure Score Monitoring:**
 - Enable and monitor Secure Score in Azure Security Center to maintain and improve security posture.
 - Implement remediation actions for identified security gaps and document improvement progress.
3. **Runbook for Off-Hours Resource Management:**
 - Create a Runbook to automatically shut down non-production resources during off-hours to optimize costs.

Acceptance Criteria

1. **Networking and High Availability:**
 - VNets are peered across regions with secure connectivity.
 - Application Gateway and Load Balancer distribute traffic effectively with WAF enabled.
2. **Security and Access Control:**
 - NSGs, Private Link, and Service Endpoints secure the environment.
 - Key Vault secrets are accessible only to resources with Managed Identity.
 - JIT and Conditional Access are enforced for administrative access.
3. **Monitoring and Alerts:**
 - Log Analytics Workspace captures logs from critical resources.
 - Custom alerts trigger notifications via Action Groups, and Logic Apps handle automated responses.
4. **Backup and Compliance:**
 - Automated backups are configured, with periodic verification.

- Azure Policy and Secure Score are actively monitored with documented compliance.
- 5. **Governance and Cost Optimization:**
 - Resources are tagged as per policy, and non-production resources shut down during off-hours.

Expected Outcome

DataFortify Inc. will have a highly secure, automated, and compliant Azure environment with:

1. **Scalability:** The infrastructure is designed for multi-region deployment and scalability.
2. **Enhanced Security:** Managed Identities, Private Link, NSGs, and Conditional Access protect resources.
3. **Real-Time Monitoring:** Comprehensive monitoring with custom alerts for quick issue detection and response.
4. **Automated Backup and Compliance:** Backup, data integrity verification, and Azure Policy ensure compliance and business continuity.
5. **Cost Efficiency:** Off-hours resource management and optimized storage and networking configurations help control costs.